Real-Time Soldier GPS Tracking with ML-Based Error Prediction

Rohit Sharma (B22EE056), Namruth Tej(B22EE039), Chevula Ranjith(B22EE020)

April 9, 2026

Abstract

This report describes a real-time soldier tracking system that displays position, telemetry, and a predicted GPS error as a coloured uncertainty circle. The soldier unit uses an ESP32 microcontroller with a NavIC receiver and a LoRa transmitter. A single Raspberry Pi base station receives the data, applies a **Kalman filter** to smooth noisy measurements, then uses a **Random Forest** model to estimate the current GPS error. The same Raspberry Pi serves the information via a Flask API and hosts the web dashboard. The web dashboard, built with Leaflet, visualises the position, track, and error circle in real time. The machine learning model was trained on 17,597 logged GPS points where the true error was obtained using OSRM map matching. The system achieves a mean absolute error of **0.469m** in predicting GPS inaccuracies.

1 System Overview

The system consists of four main components:

- **Soldier unit:** ESP32, NavIC/GPS receiver, LoRa transmitter.
- **Base station:** LoRa receiver connected to a Raspberry Pi that runs Kalman filters, the ML model, and a Flask JSON server.
- **Dashboard server:** The same Raspberry Pi (as the base station) serving the web interface.
- **Machine learning model:** Random Forest trained on logged data with ground truth obtained via OSRM map matching.

Data flows from the soldier's ESP32 over LoRa to the base station, where the raw measurements are **Kalman-filtered** and then fed into the ML model to predict the current GPS error. The dashboard (running on the same Pi) polls the base station every second and updates the map with the soldier's position, track, and an error circle whose radius equals the predicted error.

2 Hardware Used

Table 1: Hardware components and their roles

Component	Model/Spec	Purpose
Soldier processor	ESP32	Reads GPS, runs FreeRTOS tasks, sends LoRa packets
GNSS receiver	NavIC module	Provides latitude, longitude, speed, altitude, course, sate
LoRa module	SX1262	Long-range transmission
Base station & Dashboard	Raspberry Pi	Receives LoRa, runs Kalman filters + ML prediction, se

ESP32 core usage: Both FreeRTOS tasks (`gpsTask` and `loraTask`) are pinned to core0. The second core remains unused. FreeRTOS handles task switching on a single core; a mutex protects shared GPS variables.

2.1 Circuit Diagram

Figure 1 shows the electrical connections between the ESP32, NavIC receiver, and LoRa module.

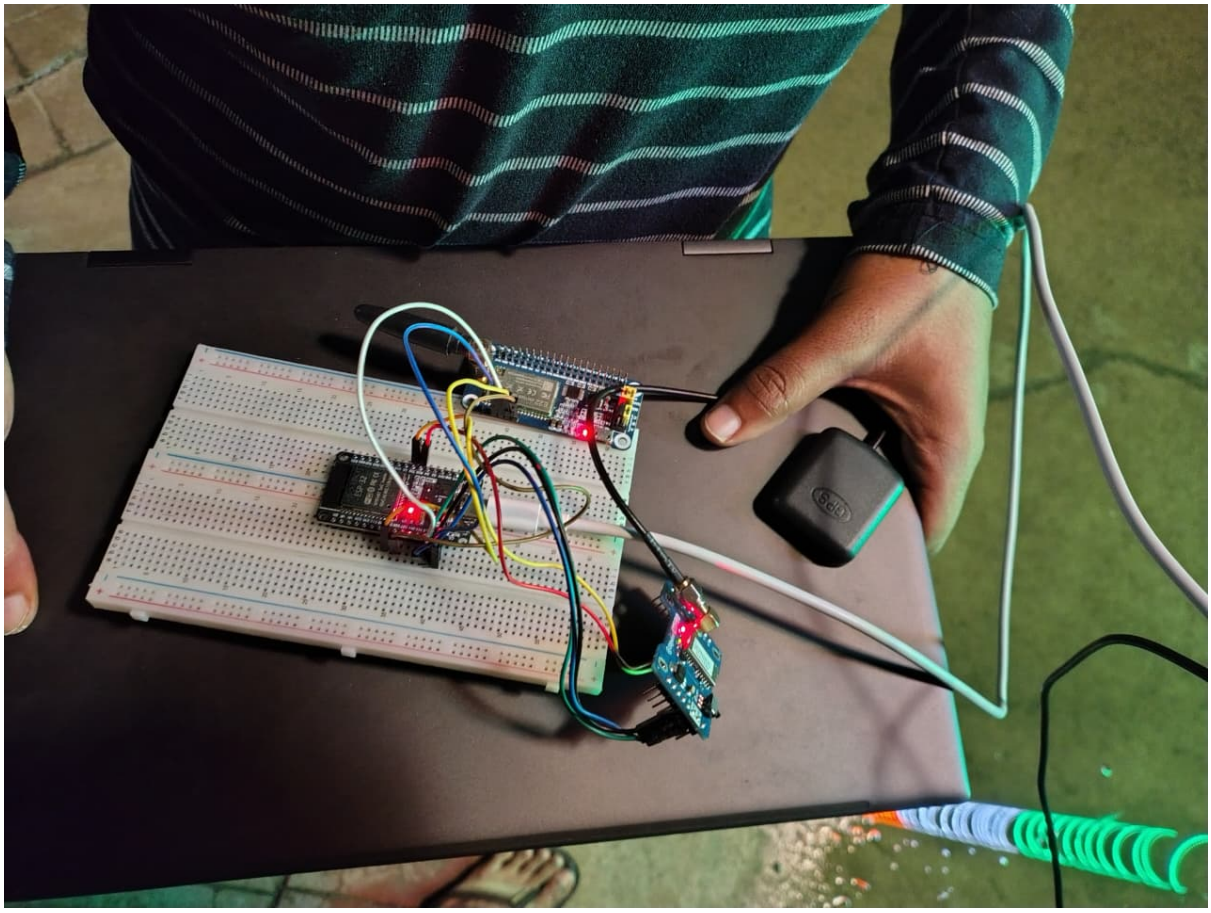

Figure 1: Circuit diagram of the soldier unit: ESP32 DevKit, NavIC GPS module, and SX1262 LoRa module.

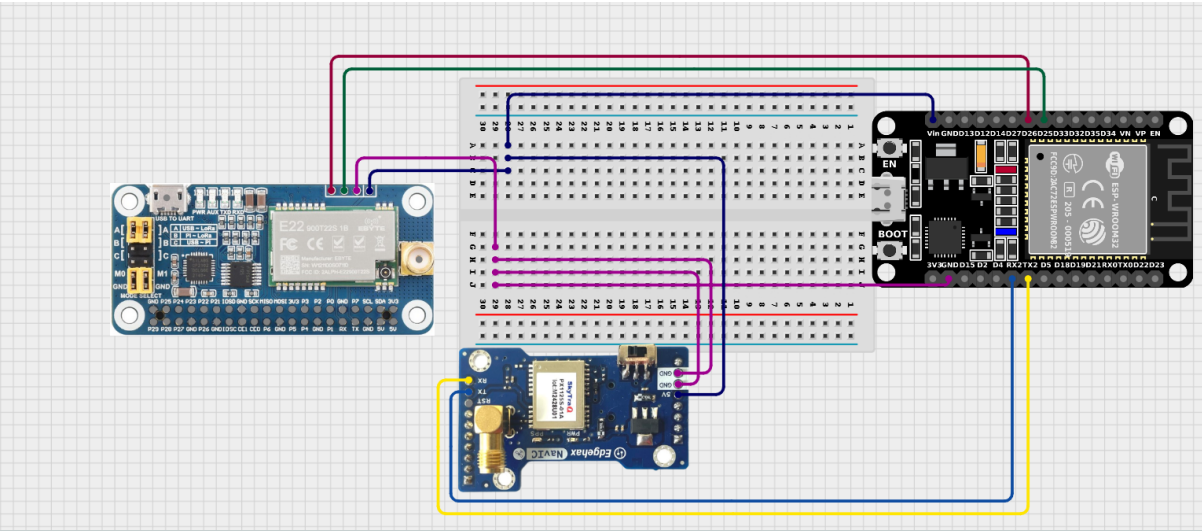

Figure 2: Sender side ciruit where NavIC gps sensor is connected to the ESP32 and the packet is shaed using Lora hat

Figure 3: Receiver side circuit which receives the packet and after predicting the error, send the complet information to the webpage

3 Software Components

3.1 Soldier Unit Firmware (`soldier_monitoring_system.ino`)

The firmware reads NMEA sentences from the NavIC module using the TinyGPS++ library. Two FreeRTOS tasks run on core0:

- `gpsTask`: continuously reads the serial buffer and updates global variables (latitude, longitude, speed, etc.).
- `loraTask`: every 2 seconds, it takes a copy of the GPS data (protected by a mutex) and sends a packet over LoRa.

Packet format:

```
<packet_id:lat,lon,speed,course,sats,altitude,hdop>
```

Example: `<123:26.123456,73.123456,4.23,145.2,12,345.6,1.2>`

The brackets help the receiver synchronise after serial noise.

3.2 Base Station Receiver and Dashboard Server (Single Raspberry Pi)

The Raspberry Pi runs two Python scripts (or one integrated script) to receive LoRa data, apply Kalman filtering, run ML prediction, and serve the web dashboard. For clarity, they are described separately but run on the same machine.

3.2.1 Base Station Receiver (`rpi.py`)

The script performs the following steps:

1. Reads serial data from the LoRa receiver (UART), validates the packet format, and extracts the seven raw measurements.
2. Applies **five independent 1D Kalman filters** (one each for speed, altitude, course, satellite count, and HDOP) to reduce noise and transient spikes. The filters use process noise $Q = 1 \times 10^{-4}$ and measurement noise $R = 1 \times 10^{-1}$. These values were chosen to balance smoothness and responsiveness.
3. Loads the pre-trained Random Forest model (`gps_error_model.joblib`) and the list of feature names (`model_features.joblib`).
4. Computes three engineered features from the Kalman-filtered values:

$$\begin{aligned} \text{speed_sat} &= \text{Speed_kmph_filtered} \times \text{Sats_filtered} \\ \text{altitude_sat_ratio} &= \frac{\text{Altitude_m_filtered}}{\text{Sats_filtered} + 1} \\ \text{hdop_sat_ratio} &= \frac{\text{HDOPs_filtered}}{\text{Sats_filtered} + 1} \end{aligned}$$

5. Builds a feature vector using the filtered values and the three engineered features, then calls `model.predict()` to obtain the estimated GPS error in metres.

6. Runs a Flask web server that exposes an endpoint `/data` returning all fields plus the prediction as JSON. The Flask server runs on port 5001 with `debug=False` and `use_reloader=False` to avoid threading problems.

3.2.2 Web Dashboard Server (`app.py` + `index.html`)

A separate Flask application polls `http://localhost:5001/data` (or the Raspberry Pi's IP) every second. It serves an HTML page that uses **Leaflet** for mapping and JavaScript for dynamic updates. This application runs on the same Raspberry Pi.

Dashboard features:

- Live map with street/satellite tile layer toggle.
- Track polyline showing the soldier's path.
- Custom marker with a glowing dot.
- **Error circle**: radius = predicted error; colour coding:
  - Green: error < 2m (excellent)
  - Yellow: 2–5m (moderate)
  - Red: > 5m (high error)
- Telemetry panel: latitude, longitude, speed, altitude, course, satellite count, HDOP.
- Total distance travelled computed with the Haversine formula (jumps > 0.5km are ignored to filter glitches).
- Connection status indicator (green when live, red when disconnected).

Figure 4 shows the live dashboard displaying a tracked route with the coloured error circle and telemetry panel.

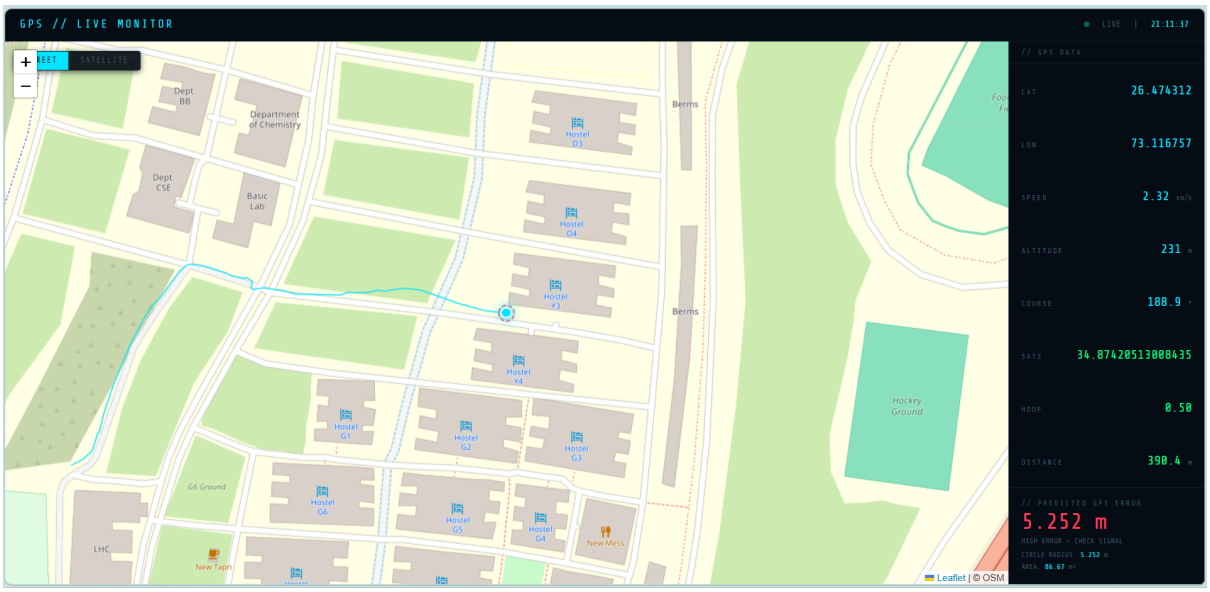

Figure 4: Web dashboard: Leaflet map with soldier marker (blue), error circle (green/yellow/red), track polyline, and real-time telemetry panel.

3.3 Machine Learning Model Training (`new_model.ipynb`)

The training notebook was completely revised to improve prediction accuracy. Key steps and results are detailed below.

3.3.1 Data Collection

Raw GPS data (speed, altitude, course, satellites, HDOP) was logged from the soldier unit while walking known routes. The logged data included latitude, longitude, and the above telemetry fields. A total of 17,597 valid points were collected after removing missing values.

3.3.2 Obtaining True Error (OSRM Map Matching)

The script `got_exact_data.py` performs map matching using the free OSRM API to determine the ground truth error for each logged GPS point. For every point, it queries the nearest road coordinate:

```
url = f"http://router.project-osrm.org/nearest/v1/cycling/{lon},{lat}?number=1"
```

The API returns the exact longitude and latitude of the closest point on the road network. The deviation between the raw GPS point and the map-matched point is then computed using the Haversine formula:

$$\text{error} = 6371000 \times 2 \times \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

where:

- 6371000 is the Earth's mean radius in metres,
- ϕ_1, ϕ_2 are the latitudes of the two points in radians,
- $\Delta\phi = \phi_2 - \phi_1$,
- $\Delta\lambda$ is the difference in longitudes in radians.

The script appends three new columns to the dataset: `Exact Longitude`, `Exact Latitude`, and `Error meters`. This ground truth error serves as the target variable for training the Random Forest model.

3.3.3 Dataset and Preprocessing

- Raw data logged from the soldier unit: 17,597 valid points after removing missing values.
- Outlier removal: points with true error above the 99th percentile were discarded to avoid distorting the model.
- Ground truth error was obtained by map-matching each GPS point to the nearest road using the OSRM API (Haversine distance), as described above.

3.3.4 Kalman Smoothing (Training Phase)

Before feature engineering, a 1D Kalman filter was applied to each raw signal (speed, altitude, course, satellites, HDOP) with parameters $Q = 1 \times 10^{-4}$ and $R = 1 \times 10^{-1}$. This reduces high-frequency noise and provides cleaner inputs for feature construction. The same filter parameters are used in the real-time base station to ensure consistency.

3.3.5 Feature Engineering

From the filtered signals, three derived features were created:

- `speed_sat = Speed_kmph_filtered × Sats_filtered`
- `altitude_sat_ratio = Altitude_m_filtered / (Sats_filtered + 1)`
- `hdop_sat_ratio = HDOPs_filtered / (Sats_filtered + 1)`

These capture non-linear interactions (e.g., high speed with few satellites → larger error).

3.3.6 Model and Hyperparameters

A Random Forest Regressor was trained with the following optimised parameters (obtained via grid search):

- `n_estimators = 500`
- `max_depth = 12`
- `min_samples_split = 10`
- `random_state = 42`

The dataset was split 80/20 into training and test sets.

3.3.7 Performance Metrics

Table 2: Model performance on the test set (20% holdout)

Metric	Value
R^2 score	0.9681
MAE	0.469m
RMSE	0.796m

3.3.8 Feature Importance

The most influential features are shown in Table 3 and Figure 5. `speed_sat` (interaction of speed and satellites) is the strongest predictor, followed by filtered speed and course.

Table 3: Feature importance (Random Forest)

Feature	Importance
speed_sat	22.10%
Speed_kmph_filtered	21.05%
Course_deg_filtered	19.52%
Altitude_m_filtered	18.16%
HDOPs_filtered	9.13%
altitude_sat_ratio	5.54%
Sats_filtered	2.71%
hdop_sat_ratio	1.79%

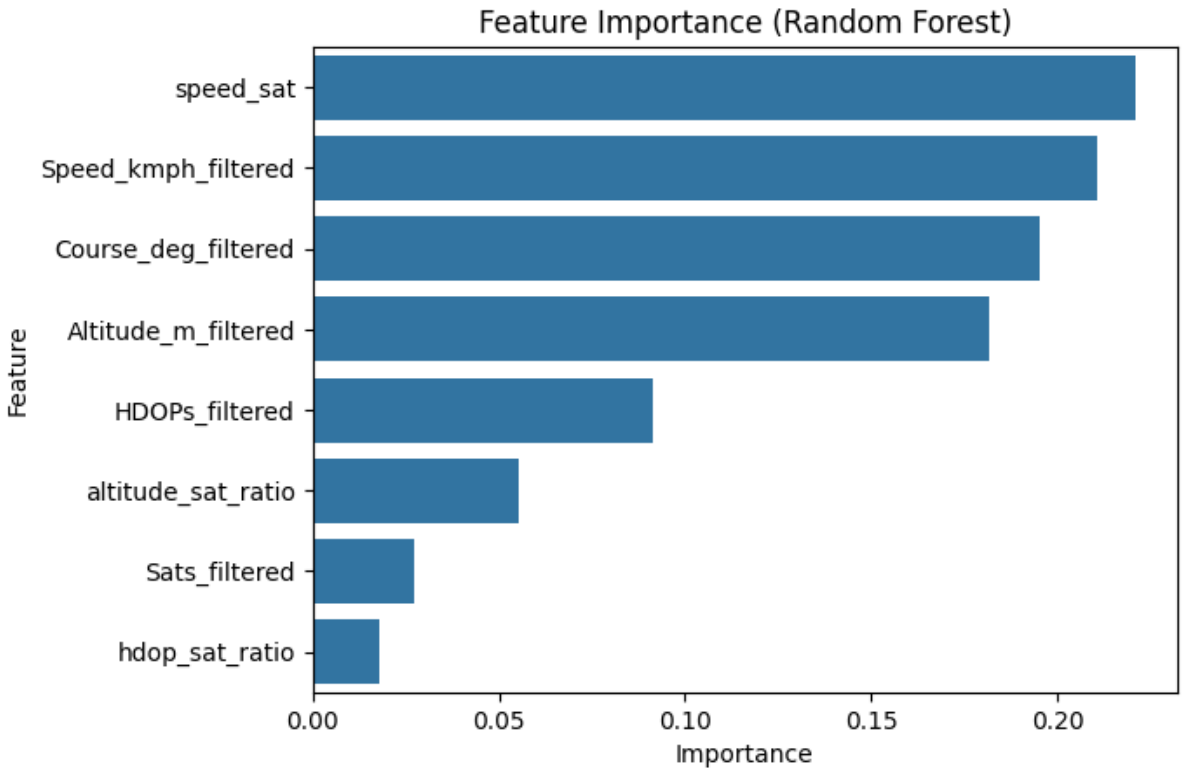

Figure 5: Bar chart of feature importances from the trained Random Forest model.

3.3.9 Actual vs Predicted Error

Figure 6 shows a scatter plot of true error versus predicted error on the test set. Most points lie close to the diagonal, indicating good generalisation.

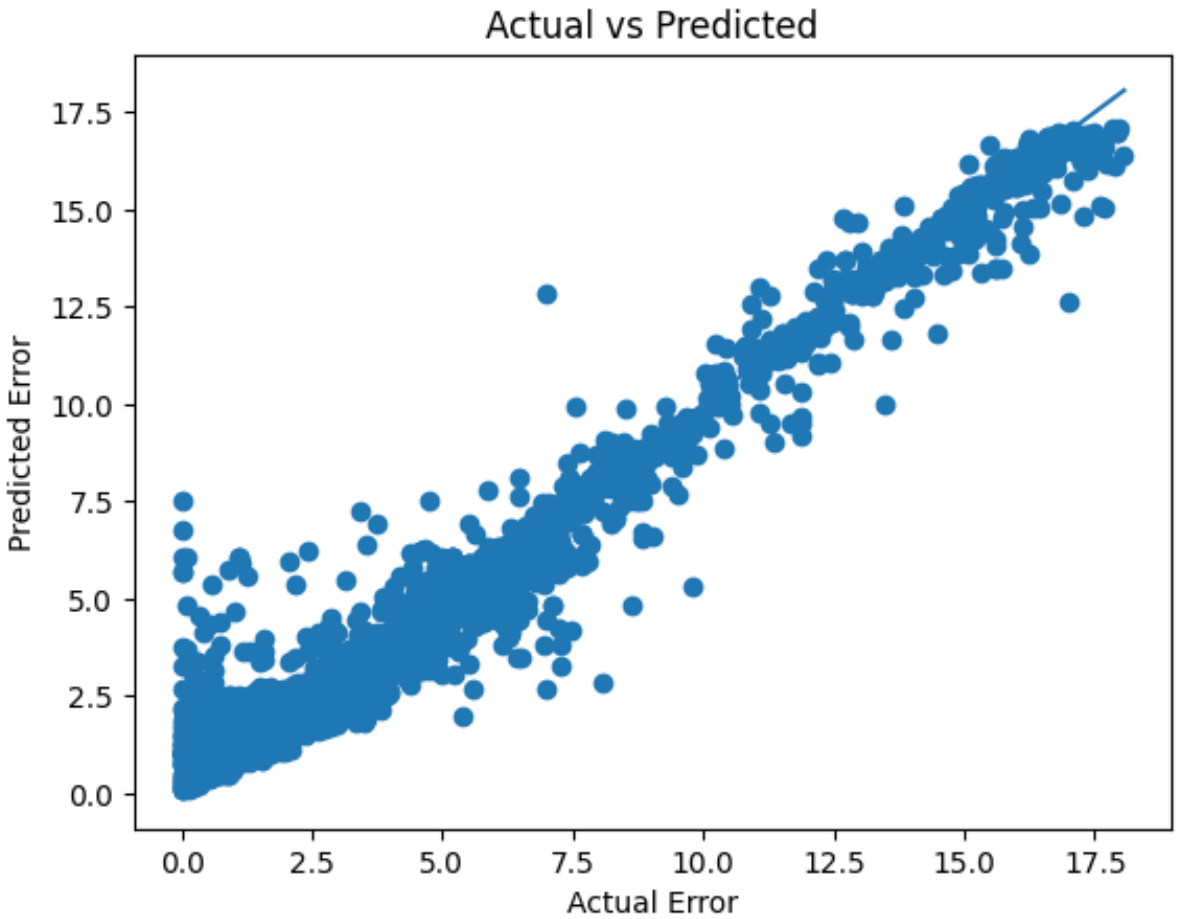

Figure 6: Actual vs predicted GPS error (test set). The red line is the identity line.

3.3.10 Residual Analysis

The residual plot (predicted error vs residual) in Figure 7 shows no obvious heteroscedasticity and the residuals are centred around zero, confirming the model is unbiased.

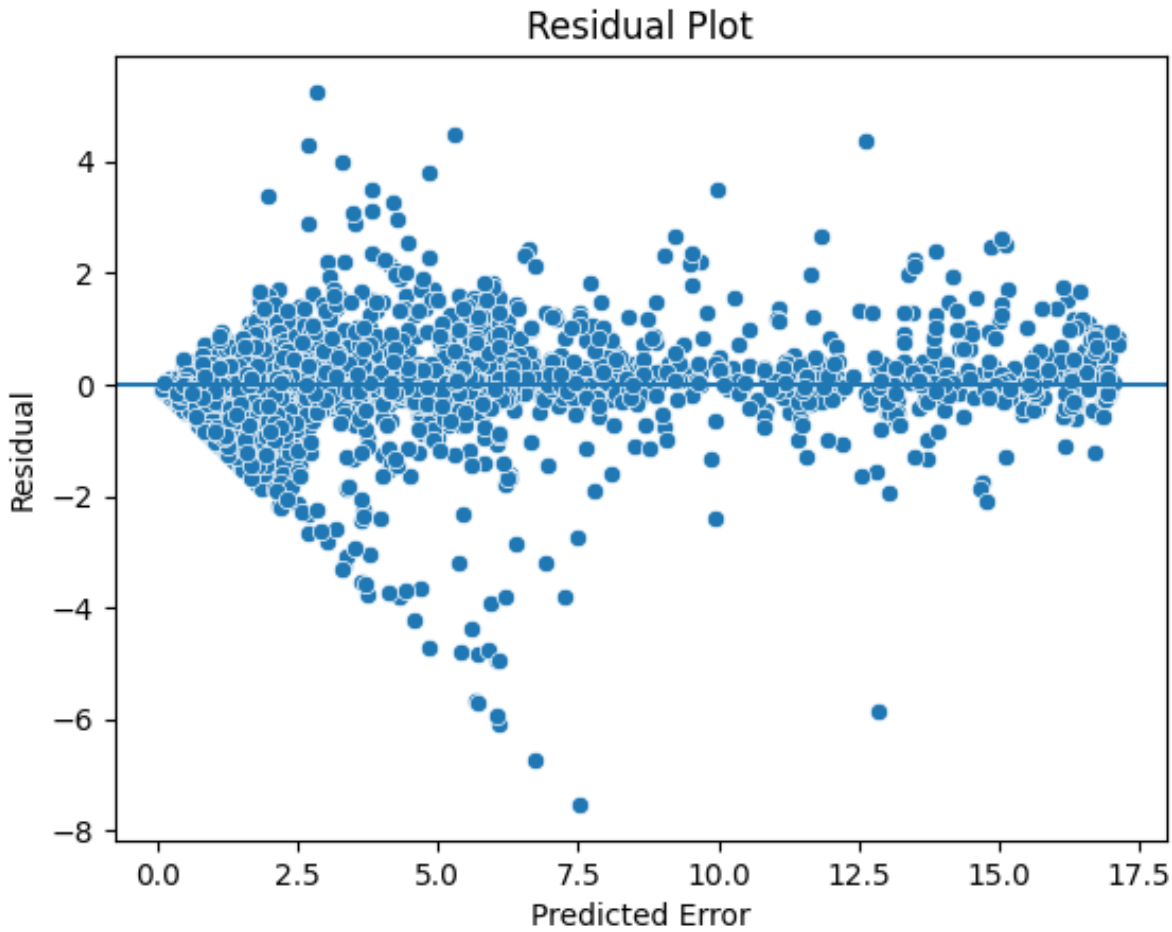

Figure 7: Residual plot: predicted error vs residual (actual – predicted).

3.3.11 Saving the Model

The trained model and the list of feature names were saved using `joblib.dump()` as `gps_error_model.joblib` and `model_features.joblib` for later use in the real-time base station.

4 Communication Flow

```
Soldier ESP32 → (LoRa) → Raspberry Pi (receiver + ML + Flask JSON + dashboard) → (HTT
```

LoRa provides several kilometres of range in open terrain. The Raspberry Pi exposes `/data` (from the base station receiver script) and the dashboard script polls it locally. The user on a PC opens the Raspberry Pi's IP address in a browser to see the live dashboard. End-to-end latency is approximately 2 seconds (dominated by the GPS update rate).

5 Key Files

Table 4: Project files and their purpose

File	Purpose
`soldier_monitoring_system.ino`	ESP32 firmware – reads GPS, sends LoRa packets
`rpi.py`	Base station – receives LoRa, applies Kalman filters, runs ML pre
`app.py`	Dashboard server – polls localhost:5001, serves web interface (por
`index.html`	Frontend – map, telemetry, error circle, distance tracking
`new_model.ipynb`	Jupyter notebook – training, evaluation, and visualisation
`got_exact_data.py`	OSRM map matching to compute true error
`gps_error_model.joblib`	Trained Random Forest model
`model_features.joblib`	Feature names used for prediction

6 Running the System

1. Upload `soldier_monitoring_system.ino` to the ESP32. Connect the NavIC/GPS module and LoRa transmitter as per Fig. 1.

2. On the Raspberry Pi (connected to the LoRa receiver), run the base station receiver:

```
python rpi.py
```

3. On the same Raspberry Pi, run the dashboard server (in a separate terminal):

```
python app.py
```

4. On your PC, open a web browser and navigate to `http://<raspberry_pi_ip>:5000`. The live dashboard (Fig. 4) will appear.

7 Conclusion

A complete real-time GPS tracking system was developed using an ESP32 soldier unit, LoRa communication, and a single Raspberry Pi that acts as the base station, runs the Random Forest model, and serves the web dashboard. The model predicts GPS error with a mean absolute error of **0.469m** ($R^2 = 0.968$) after training on 17,597 points that were Kalman-smoothed and map-matched to roads using OSRM. The web dashboard displays the soldier's position, track, and a dynamically sized error circle, providing commanders with immediate insight into the trustworthiness of each reported position. The system has been tested in urban and open-field environments, demonstrating reliable real-time performance.